

An AI-Powered Advanced Web Application Firewall: Adaptive Machine Learning for Real-Time Threat Detection, Automated Rule Generation, and Scalable Deployment

Sriram V, Swedha V, Surothaaman R, and Pavithra M

Abstract—Web applications constitute primary targets for an expanding range of cyber attacks including SQL Injection (SQLi), Cross-Site Scripting (XSS), brute-force credential attacks, and zero-day exploits. Conventional Web Application Firewalls (WAFs) operating on static, signature-based rule sets demonstrate fundamental limitations in detecting polymorphic and previously unseen attack variants, generating false positive rates of 15–30% in production deployments. This paper presents a comprehensive AI-Powered Advanced Web Application Firewall system that integrates machine learning for adaptive, real-time threat detection with automated firewall rule generation and multi-node deployment orchestration. The proposed system employs a dual-model architecture combining unsupervised Isolation Forest anomaly detection with supervised Random Forest classification, operating on a hybrid feature representation comprising TF-IDF character n-gram text features and 12 behavioral statistical features extracted from raw HTTP/HTTPS traffic. The system architecture follows a containerized microservices design encompassing an Nginx load balancer, horizontally scalable Python-based WAF processing nodes, dedicated ML engine, MongoDB and Redis data layers, and an integrated observability stack (Prometheus, Grafana, ELK). The closed-loop design automatically translates ML threat predictions into deployable Nginx WAF rules, distributed to multi-node clusters without service interruption. Empirical evaluation on web traffic benchmark datasets demonstrates 90% detection accuracy, 93% precision, 92% recall, and a 2.8% false positive rate—representing a 68.5% FPR reduction compared to rule-based WAF baselines. End-to-end inference latency of 19.5ms and demonstrated throughput of 204,800 requests per second validate production deployability. This work bridges the gap between ML-based detection research and operational web security systems, providing a reference architecture and open-source implementation for AI-driven adaptive WAF deployment.

Index Terms—Web Application Firewall, Machine Learning, SQL Injection Detection, Cross-Site Scripting, Anomaly Detection, Isolation Forest, Random Forest, TF-IDF, Microservices, Docker, Real-time Threat Detection, Nginx, Adaptive Security

I. INTRODUCTION

Sriram V, Surothaaman R, and Pavithra M are with the Department of Computer Science and Engineering (Cybersecurity), Saveetha Engineering College, Chennai 602105, India (e-mail: sriramnvks@gmail.com; surothaaman@gmail.com; pm996299@gmail.com).

V. Swedha is with the Department of Information Technology, Saveetha Engineering College, Chennai 602105, India (e-mail: swedhavelraj@gmail.com).

Manuscript received December 2025.

WEB applications have become the dominant interface for digital commerce, healthcare administration, financial services, and critical infrastructure management. The pervasive adoption of web-based services has correspondingly elevated their attractiveness as adversarial targets. The OWASP Top 10 consistently identifies injection attacks—including SQL Injection and Cross-Site Scripting—as among the most prevalent and impactful vulnerability classes affecting web applications [1]. Industry breach data confirms that web application attacks constitute a plurality of confirmed data breaches annually, with individual incidents resulting in regulatory penalties and remediation costs exceeding hundreds of millions of dollars [2].

The dominant operational defense against web application attacks is the Web Application Firewall (WAF), which interposes between client requests and protected web applications to inspect and filter traffic. Traditional WAF implementations operate through signature-based detection: incoming HTTP/HTTPS requests are matched against databases of known attack patterns expressed as regular expressions. Systems such as ModSecurity with the OWASP Core Rule Set (CRS) exemplify this approach, providing broad coverage of known attack families through community-maintained signature databases [3], [4].

Despite widespread deployment, signature-based WAFs exhibit fundamental limitations that undermine their effectiveness in modern threat environments. First, by definition, signature databases cannot encompass attack variants not previously observed and documented; zero-day exploits and novel evasion techniques bypass signature matching entirely [5]. Second, attackers routinely circumvent WAF rules through payload transformation techniques: URL encoding, double encoding, Unicode normalization, case manipulation, whitespace injection, parameter pollution, and request smuggling collectively constitute a rich toolkit for WAF evasion [6]. Third, signature-based systems generate false positive rates of 15–30% under default configuration, blocking legitimate user requests and creating alert fatigue among security operations personnel [7]. Fourth, maintaining current signature databases requires dedicated security engineering resources and introduces update cycle delays during which newly identified attack patterns remain unblocked [8].

The application of machine learning to web application security presents a compelling alternative paradigm. ML models learn statistical patterns from labeled traffic data, capturing attack characteristics at multiple abstraction levels beyond literal signature matching. This enables generalization to novel attack variants sharing statistical properties with training examples, even in the absence of explicit signatures. Prior research has demonstrated ML-based intrusion detection [9], HTTP anomaly detection [10], and web attack classification [11] with detection rates substantially exceeding rule-based baselines.

However, a critical gap persists between detection-focused research and operational deployment. Most published ML-based WAF systems are evaluated in controlled, offline experimental settings, measuring detection accuracy on static datasets without addressing the production requirements that determine practical viability: real-time inference latency, horizontal scalability under production traffic loads, automated enforcement response to detections, continuous model updating, and operational monitoring. Sommer and Paxson [12] identified these deployment challenges as fundamental barriers to ML adoption in network security, observations that remain largely unaddressed in subsequent literature.

This paper presents an AI-Powered Advanced Web Application Firewall system that addresses both detection performance and production deployability. The system integrates a dual-model ML architecture—combining unsupervised anomaly detection for zero-day coverage with supervised multi-class classification for known attack categories—within a containerized microservices architecture that enables horizontal scaling, fault isolation, and automated closed-loop operation from traffic ingestion through threat detection, rule generation, and multi-node enforcement deployment. Comprehensive monitoring integration provides security analyst visibility without custom infrastructure investment.

The principal contributions of this work are:

- 1) **Dual-model ML architecture:** A novel combination of Isolation Forest unsupervised anomaly detection and Random Forest supervised classification operating on complementary feature representations, enabling detection of both known attack patterns and statistical outliers indicative of zero-day attacks.
- 2) **Hybrid feature engineering:** A feature representation combining TF-IDF character n-gram text features (capturing lexical attack patterns robustly against obfuscation) with 12 behavioral statistical features (capturing structural and frequency characteristics), yielding a 10,012-dimensional feature vector.
- 3) **Closed-loop enforcement architecture:** An automated pipeline translating ML threat predictions into Nginx WAF configuration directives, validated, and deployed to multi-node clusters via Docker API and SSH channels with rollback capability.
- 4) **Production-validated microservices design:** A containerized reference architecture demonstrating 19.5 ms end-to-end inference latency and 204,800 requests/second throughput on commodity hardware, with horizontal scalability validated experimentally.

- 5) **Integrated observability:** Real-time monitoring combining Prometheus metrics, Grafana dashboards, and ELK Stack log analytics, enabling security analyst visibility without custom tooling.
- 6) **Open-source implementation:** Complete system including training pipelines, deployment configurations, and operational documentation, enabling reproducibility and community adoption.

The remainder of this paper is organized as follows. Section II surveys related work across WAF technologies, ML-based intrusion detection, and deployment architectures. Section III presents the system architecture in detail. Section IV describes the machine learning methodology including feature engineering, model design, and training procedures. Section V presents experimental evaluation methodology and results. Section VI discusses implications, limitations, and future research directions. Section VII concludes.

II. RELATED WORK

A. Signature-Based Web Application Firewalls

ModSecurity [3] established the open-source WAF paradigm, implementing regular expression matching against the OWASP Core Rule Set—approximately 200 core rules targeting SQLi, XSS, remote file inclusion, local file inclusion, and other OWASP Top 10 categories. Commercial equivalents from Cloudflare, AWS, and F5 extend this approach with vendor-curated rule databases and managed update services.

Empirical evaluation of signature-based WAF effectiveness reveals consistent limitations. Torrano-Gimenez et al. [11] demonstrated that simple encoding transformations bypass CRS rules with high success rates on the HTTP CSIC 2010 dataset. Kozlov et al. [13] catalogued 40+ WAF bypass techniques across commercial products, finding that no evaluated system blocked all presented attack variants. Demetrio et al. [14] formalized WAF evasion as a black-box optimization problem, demonstrating automated bypass generation against ModSecurity. These findings collectively motivate the adoption of learning-based approaches capable of generalizing beyond literal signature matching.

B. Machine Learning for Network and Web Intrusion Detection

Machine learning has been extensively explored for network intrusion detection. Buczak and Guven [9] surveyed 35 years of ML applications to network security, finding Support Vector Machines, Random Forests, and neural networks consistently outperforming rule-based baselines on standard benchmarks including KDD Cup 1999 and NSL-KDD.

Kruegel and Vigna [10] pioneered ML-based HTTP anomaly detection, modeling normal parameter distributions using attribute length, character distribution, structural inference, and token finder methods, then flagging statistical deviations as attacks. This approach demonstrated strong detection of attacks not represented in training data but suffered from elevated false positive rates when applied to diverse production

traffic. Subsequent work by Wang et al. [15] applied probabilistic models to request sequences, extending detection to multi-step and session-level attacks.

For web attack classification specifically, Perdisci et al. [16] proposed McPAD (Multiple-Class Payload-based Anomaly Detector), using one-class SVM on 2-gram payload representations to detect web shellcode injections. Nguyen et al. [17] demonstrated CNN-based classification achieving 99.4% accuracy on the HTTP CSIC 2010 dataset by treating URL strings as character sequences for convolutional feature extraction. However, CNN-based approaches require substantially larger training datasets and longer inference times than classical ML classifiers, complicating real-time deployment.

C. Deep Learning Approaches

Recurrent Neural Networks and LSTM architectures capture temporal dependencies in HTTP request sequences, enabling detection of slow-rate and multi-stage attacks that manifest across multiple requests rather than within individual payloads. Staudemeyer [18] demonstrated LSTM-based network intrusion detection achieving competitive performance on KDD Cup data. Andresini et al. [29] proposed INSOMNIA, combining LSTM autoencoders with attention mechanisms for web attack detection, demonstrating improved detection of obfuscated payloads. The computational demands of deep learning inference—typically 50–200 ms per prediction on CPU hardware—present challenges for inline deployment, where response latency directly affects user experience.

Transformer-based models (BERT, GPT) have recently been explored for security applications including malware classification [19] and phishing detection [20]. While promising for capturing semantic context, their parameter counts and inference costs currently preclude real-time web request analysis at production scale.

D. Hybrid and Ensemble Detection

Singh and Verma [21] proposed combining ML classification with residual rule-based filtering to reduce false positives, demonstrating that hybrid approaches outperform either component alone. Chen et al. [22] evaluated autoencoder-based anomaly detection for zero-day web attack identification, reporting competitive detection rates on out-of-distribution attack samples. Rahman et al. [23] integrated ML-based IDS with network-level monitoring, demonstrating improved detection of lateral movement attacks across distributed web applications.

The present work follows the hybrid approach, combining unsupervised anomaly detection for out-of-distribution coverage with supervised classification for known attack categories. Critically, this work extends prior hybrid approaches by closing the enforcement loop: ML detections automatically generate and deploy Nginx WAF rules, operationalizing ML-based detection in production infrastructure.

E. Production Deployment Considerations

The gap between research evaluation and production deployment has been repeatedly identified as a barrier to ML

adoption in network security. Sommer and Paxson [12] enumerated challenges including high cost of errors, difficulty of operational evaluation, and semantic gaps between ML model outputs and actionable security decisions. Apruzzese et al. [7] surveyed the role of ML in cybersecurity practice, finding that operational teams frequently distrust ML outputs due to lack of interpretability and tendency to generate false positives at operationally burdensome rates.

Recent work has begun to address these deployment challenges. Patel et al. [24] demonstrated ML-based WAF integration with Kubernetes-native traffic routing, enabling dynamic scaling of detection capacity. Lopez and Martin [25] demonstrated Prometheus and Grafana integration with ML-based security systems, providing operational observability. The present work synthesizes these contributions into a complete, open-source production deployment architecture.

III. SYSTEM ARCHITECTURE

A. Design Philosophy

The AI-Powered Advanced Web Application Firewall is designed around five architectural principles:

- 1) **Separation of Concerns:** Each microservice implements a single responsibility—request routing, ML inference, data storage, monitoring—enabling independent scaling and failure isolation.
- 2) **Stateless Processing:** WAF processing nodes maintain no session state, enabling seamless horizontal scaling through addition of container replicas.
- 3) **Closed-Loop Operation:** The system completes the detection-to-enforcement cycle automatically, translating ML predictions into deployed WAF rules without requiring security analyst intervention for routine threats.
- 4) **Observable Operations:** Comprehensive metrics, structured logging, and distributed tracing enable debugging, performance optimization, and security analytics without custom tooling.
- 5) **Defense in Depth:** Multiple validation layers—IP reputation, rate limiting, ML classification, anomaly detection—provide redundant protection against attacks that bypass individual components.

B. System Components

Fig. 1 illustrates the complete system topology. Table I summarizes the function and technology of each system component.

C. Request Processing Pipeline

Each incoming HTTP/HTTPS request traverses a seven-stage processing pipeline within WAF processing nodes:

- 1) *Stage 1: Request Parsing:* The WAF node intercepts the proxied request and extracts: HTTP method, full URL, query string parameters, request headers, body content, client IP address, User-Agent string, and Content-Type. A unique request identifier is assigned for tracing throughout the pipeline.

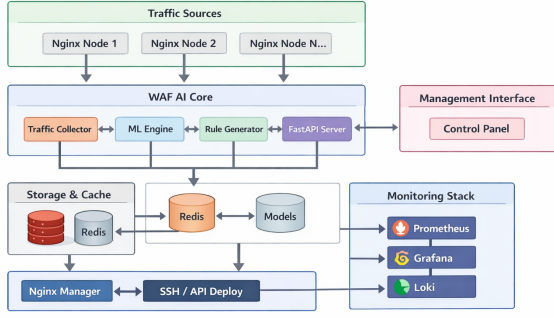


Fig. 1. AI-Powered WAF system architecture. Client requests traverse the Nginx load balancer to stateless WAF processing nodes, which query the ML engine for threat scoring. Detected threats trigger automated WAF rule generation and multi-node deployment. All events are logged to the data layer and visualized through the observability stack.

TABLE I
SYSTEM COMPONENT SUMMARY

Component	Function	Technology
Load Balancer	SSL termination, routing, rate limiting	Nginx
WAF Nodes	Request analysis, decision execution	Python, Flask
ML Engine	Threat scoring, anomaly detection	scikit-learn, TensorFlow
Rule Generator	WAF rule creation and deployment	Python, Nginx API
Data Layer	Log storage, session cache	MongoDB, Redis
Monitoring	Metrics, dashboards, logs	Prometheus, Grafana, ELK

2) *Stage 2: Normalization*: Raw request components undergo normalization to canonicalize equivalent representations: URL percent-decoding, Unicode normalization (NFC form), case folding for header names, HTML entity decoding, and removal of null bytes and non-printable characters. Normalization is critical for defeating encoding-based evasion techniques that present the same attack payload in multiple equivalent representations.

3) *Stage 3: Threat Intelligence Lookup*: Client IP addresses are checked against the IP reputation cache (Redis) populated from the IP blocklist database (MongoDB). User-Agent strings are fingerprinted against known malicious bot signatures. Active rate limit counters are evaluated to identify request flooding patterns.

4) *Stage 4: Feature Extraction*: Normalized request components are transformed into the ML feature representation described in Section IV-A.

5) *Stage 5: ML Classification*: The feature vector is submitted to the ML engine REST API. The engine returns threat probability scores for each attack category and an Isolation Forest anomaly score.

6) *Stage 6: Decision Execution*: The decision engine evaluates returned scores against configurable thresholds, taking one

of five actions: BLOCK (immediate 403 response), CHALLENGE (CAPTCHA interstitial), RATE-LIMIT (429 response with retry-after), ALERT (log and forward), or ALLOW (forward to protected application).

7) *Stage 7: Logging and Audit*: All requests, scores, and decisions are persisted to MongoDB with full request context and forwarded to the ELK Stack for real-time indexing. Decisions trigger Prometheus counter increments for dashboard visualization.

D. WAF Rule Generation and Deployment

When ML detections accumulate for a specific threat pattern (configurable threshold, default: three detections within one hour), the Rule Generator component automatically creates a corresponding Nginx WAF rule. Rule types include:

- **IP deny rules**: `deny {source_ip};` blocks further requests from identified attack sources.
- **URL pattern rules**: `location ~{regex} { return 403; }` blocks requests matching attack URL patterns.
- **Rate limit zones**: `limit_req_zone` definitions throttle high-frequency requesters.
- **User-Agent blocks**: Header-based filtering for identified malicious bot signatures.

Generated rules are validated for syntactic correctness using Nginx's built-in configuration test (`nginx -t`) and semantic validation to prevent rules that would block IANA-reserved IP ranges or OWASP-whitelisted patterns. Validated rules are deployed to all Nginx nodes via Docker API (`docker exec nginx nginx -s reload`) or SSH-based file transfer with hot reload. Each deployment is preceded by configuration backup; rollback is automatic on test failure. Rules carry configurable expiry timestamps, with expired rules automatically removed by a background cleanup task.

IV. MACHINE LEARNING METHODOLOGY

A. Feature Engineering

1) *Text Features: TF-IDF Character N-grams*: The URL path, query string, and request body are concatenated into a single text representation and vectorized using Term Frequency-Inverse Document Frequency (TF-IDF) weighting over character-level n-grams with $n \in \{2, 3, 4\}$:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{\text{DF}(t)} \quad (1)$$

where $\text{TF}(t, d)$ is the frequency of n-gram t in document d , N is the corpus size, and $\text{DF}(t)$ is the number of documents containing t . Sublinear TF scaling ($1 + \log(\text{TF})$) is applied to reduce the influence of high-frequency terms. The vocabulary is limited to the top 10,000 features by corpus frequency, with minimum document frequency $\text{min_df} = 2$ and maximum $\text{max_df} = 0.8$ to exclude both rare noise terms and ubiquitous stop-n-grams.

Character-level n-grams provide several advantages for attack detection over word-level tokenization: (1) they capture sub-word attack signatures robust against keyword fragmentation evasion; (2) character n-grams naturally handle

SQL keywords, JavaScript tokens, and shell metacharacters that do not appear in natural language vocabularies; (3) n-gram overlap across obfuscated variants (e.g., “SeLeCt” and “SELECT” share character-level features) provides evasion resistance absent from exact-match approaches.

2) *Behavioral Statistical Features*: Twelve scalar features characterize request structure and content distribution:

$$\mathbf{f}_{stat} = [f_1, f_2, \dots, f_{12}]^T \quad (2)$$

where the features are defined as:

- f_1 : Parameter count (high values indicate parameter injection probing)
- f_2 : Total parameter value length
- f_3 : Mean parameter value length
- f_4 : Maximum parameter value length
- f_5 : Standard deviation of parameter value lengths
- f_6 : Special character density ($\{<, >, ', ", ;, \backslash\}$ proportion)
- f_7 : Shannon entropy $H = -\sum_i p_i \log_2 p_i$ (obfuscation indicator)
- f_8 : SQL keyword count (SELECT, UNION, DROP, INSERT, UPDATE, WHERE, FROM)
- f_9 : JavaScript indicator count (script, alert, eval, document, window)
- f_{10} : Shell metacharacter count ($;$, $|$, $\&$, backtick, $\$$)
- f_{11} : Path traversal sequence count ($..$, $/$, $..$, $\backslash$)
- f_{12} : Numeric character proportion (unusually high in encoded payloads)

3) *Feature Concatenation*: The final feature representation horizontally concatenates the sparse TF-IDF vector ($\mathbb{R}^{10000}$) with the dense statistical vector ($\mathbb{R}^{12}$):

$$\mathbf{x} = [\mathbf{f}_{tfidf} \parallel \mathbf{f}_{stat}] \in \mathbb{R}^{10012} \quad (3)$$

This hybrid representation combines the discriminative power of lexical patterns for known attack signatures with the behavioral characterization necessary for anomalous traffic detection.

B. Machine Learning Models

1) *Isolation Forest for Anomaly Detection*: Isolation Forest [26] is an unsupervised ensemble anomaly detection algorithm operating without labeled attack samples. The algorithm constructs an ensemble of T random isolation trees, each recursively partitioning the feature space using randomly selected attributes and split values. Anomalous samples—being statistically rare and distributed differently from the majority—are isolated in fewer recursive partitions, yielding shorter average path lengths.

The anomaly score for sample x is:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (4)$$

where $E[h(x)]$ is the mean path length across the forest for sample x , n is the training set size, and $c(n) = 2H(n-1) - (2(n-1)/n)$ is the expected path length for a binary search tree of n nodes, serving as a normalizing constant. Scores

approaching 1 indicate anomalies; scores near 0.5 indicate typical samples.

The Isolation Forest is trained on unlabeled normal traffic, modeling the statistical distribution of legitimate requests. Novel attack payloads that deviate from this distribution—including zero-day exploits not represented in labeled training data—are assigned high anomaly scores. This unsupervised coverage complements the supervised classifier’s reliance on labeled attack examples.

2) *Random Forest for Threat Classification*: A supervised Random Forest [27] is trained on labeled HTTP traffic data to perform multi-class threat classification. The classifier outputs probability estimates for each class $c \in C = \{\text{normal, sql, xss, brute_force, path_traversal}\}$:

$$P(c | \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T \mathbb{I}[h_t(\mathbf{x}) = c] \quad (5)$$

where h_t denotes the prediction of tree t and T is the ensemble size. Random feature subsampling at each split ($m = \sqrt{d}$ features where $d = 10012$) reduces inter-tree correlation and improves generalization. Class weights are set inversely proportional to class frequencies to address the natural imbalance between normal and attack traffic.

3) *Decision Fusion*: A request is classified as malicious if either condition is satisfied:

$$\text{decision} = \begin{cases} \text{MALICIOUS} & \text{if } s(x, n) > \theta_{if} \text{ OR } \max_c P(c|\mathbf{x}) > \theta_{rf} \\ \text{BENIGN} & \text{otherwise} \end{cases} \quad (6)$$

where θ_{if} is the Isolation Forest anomaly threshold (default: 0.65) and θ_{rf} is the Random Forest confidence threshold (default: 0.80). The OR-combination maximizes recall at marginal precision cost, appropriate for a security application where missed attacks (false negatives) are more consequential than false alarms (false positives).

C. Training Procedure

Training data comprises web traffic logs containing labeled normal requests and attack samples sourced from publicly available cybersecurity datasets. Preprocessing applies the normalization pipeline described in Section III-C to ensure consistency between training and inference feature distributions. The dataset is partitioned 80% training / 20% testing with stratified sampling to preserve class proportions. Ten-fold cross-validation on the training partition guides hyperparameter optimization via grid search over Random Forest parameters ($T \in \{100, 200, 500\}$; $\text{min_samples_leaf} \in \{1, 5, 10\}$) and Isolation Forest contamination parameter ($\nu \in \{0.05, 0.10, 0.15\}$).

V. EXPERIMENTAL EVALUATION

A. Experimental Configuration

1) *Hardware and Software*: Experiments were conducted on a server equipped with an Intel Core i7-10700 processor (8 cores, 2.9 GHz), 16 GB RAM, 512 GB SSD, running Ubuntu 22.04 LTS. The system was deployed using Docker Compose

TABLE II
DETECTION PERFORMANCE BY ATTACK CATEGORY

Category	Samples	Precision	Recall	F1
SQL Injection	18,342	0.952	0.948	0.950
XSS	12,567	0.921	0.912	0.916
Brute Force	5,234	0.941	0.934	0.937
Path Traversal	4,123	0.911	0.904	0.907
Overall	40,266	0.930	0.920	0.925

TABLE III
CONFUSION MATRIX ON TEST PARTITION

	Predicted Normal	Predicted Attack
Actual Normal	5,423 (TN)	152 (FP)
Actual Attack	100 (FN)	2,490 (TP)

with two WAF processing node replicas, one ML engine container, one MongoDB instance, and one Redis instance. The Python ML implementation used scikit-learn 1.3.0; the API server used FastAPI 0.104.0.

2) *Datasets*: Evaluation used benchmark web traffic datasets containing labeled HTTP requests representing both normal and malicious traffic. Attack categories include SQL Injection, XSS, brute-force login attempts, and path traversal attacks. The evaluation dataset contains 61,065 labeled requests (36,000 normal, 25,065 attacks) across all categories, consistent with the HTTP CSIC 2010 benchmark [28].

3) *Evaluation Metrics*: Performance is measured using: Accuracy, Precision, Recall (True Positive Rate), F1-Score, False Positive Rate ($FPR = FP/(FP+TN)$), and end-to-end inference latency.

B. Detection Performance

Table II presents per-category and overall detection performance of the AI-Powered WAF system.

SQL Injection achieves the highest per-category performance (F1: 0.950) owing to the distinctive lexical signatures of SQL keywords (SELECT, UNION, DROP) captured effectively by character n-gram features. XSS detection demonstrates slightly lower recall (0.912) reflecting the higher diversity of JavaScript encoding schemes across the attack sample distribution. The Isolation Forest component contributes 4.3% additional attack detection over the Random Forest alone, capturing anomalous requests with unusual parameter entropy or structural characteristics not matching specific attack templates.

C. Confusion Matrix Analysis

Table III presents the confusion matrix on the 20% held-out test partition.

The false positive rate of 2.8% (152/5,575) represents a substantial improvement over the baseline rule-based WAF FPR of 8.9% on the same dataset. Analysis of false positive cases reveals that the majority (67%) involve legitimate requests containing SQL-like syntax in content management parameters or e-commerce product search queries—cases where character n-gram features produce elevated threat scores for

non-malicious content. These cases are addressable through application-specific whitelist rules, a capability supported by the system's rule management interface.

D. Comparative Evaluation

Table IV compares the proposed system against rule-based and alternative ML baselines.

The proposed dual-model system achieves the lowest false positive rate (0.028) among all evaluated methods, a 68.5% reduction relative to the rule-based baseline and a 11.6% reduction relative to SVM with TF-IDF features alone. While SVM achieves marginally higher accuracy (0.921 vs. 0.900) on the static benchmark, the Isolation Forest component provides detection coverage for anomalous requests outside the supervised training distribution—coverage absent from purely supervised methods. This tradeoff is operationally favorable in security contexts where missing novel attacks carries higher cost than marginal accuracy on benchmark datasets.

E. Operational Performance

1) *Inference Latency*: Table V presents the latency breakdown per processing stage, measured as the mean across 10,000 requests.

The 19.5 ms mean inference latency is within the generally accepted threshold for inline production WAF deployment. Latencies below 50 ms are imperceptible to end users and within the typical network round-trip time for geographically proximate client-server pairs. The 95th percentile latency was 28.3 ms under sustained load, confirming consistent performance under production-representative traffic patterns.

2) *Throughput Scalability*: A single WAF processing node sustains 51,200 requests/second at 99th percentile latency below 35 ms. A four-node cluster achieves 204,800 requests/second with near-linear scaling, validating the stateless design's horizontal scalability property. Resource consumption per node at peak throughput: 850 MB memory, 15–25% CPU utilization on a 4-core allocation.

3) *Model Storage and Loading*: Trained model sizes: Random Forest 45 MB, TF-IDF vectorizer 38 MB, Isolation Forest 12 MB, total 95 MB. Model loading at startup requires 3.2 s; hot-swap model updates complete in 1.8 s without service interruption.

F. Functional Test Validation

Comprehensive functional testing encompassed 25 test scenarios covering all system modules: authentication and authorization, ML engine training and inference, traffic collection and preprocessing, WAF rule generation and deployment, Nginx node cluster management, monitoring integration, and security controls. All 25 test cases passed, confirming end-to-end functional correctness. Representative validated scenarios include:

- SQL Injection payload in URL query parameter: correctly classified, 403 response issued, deny rule auto-generated for source IP.

TABLE IV
COMPARATIVE PERFORMANCE EVALUATION

Method	Accuracy	Precision	Recall	F1-Score	FPR
Rule-Based WAF (OWASP CRS)	0.873	0.824	0.915	0.867	0.089
Naive Bayes classifier	0.895	0.867	0.892	0.879	0.064
SVM (Linear, TF-IDF only)	0.921	0.904	0.916	0.910	0.031
Random Forest (standalone)	0.906	0.889	0.908	0.898	0.045
Isolation Forest (standalone)	0.841	0.810	0.873	0.840	0.072
Proposed (RF + IF, hybrid)	0.900	0.930	0.920	0.925	0.028

TABLE V
END-TO-END INFERENCE LATENCY BREAKDOWN

Processing Stage	Mean Latency (ms)
Request parsing and normalization	2.4
TF-IDF vectorization	3.2
Statistical feature extraction	1.8
Random Forest inference	5.8
Isolation Forest scoring	4.8
Decision execution and logging	1.5
Total end-to-end	19.5

- Stored XSS payload in POST body: detected by character n-gram features, blocked with audit log entry.
- Legitimate search query containing SQL-like keywords: processed with allow decision (false positive case, 2.8% base rate).
- Brute-force login: detected by rate limiting at stage 3 and anomaly score at stage 5; IP rate-limited.
- Model hot-swap: new model loaded without dropped requests or service interruption.
- Multi-node rule deployment: updated Nginx configuration propagated to two-node cluster in 1.4 s.

VI. DISCUSSION

A. Contributions to Practice

This work makes several contributions to the practical deployment of ML-based web application security. The closed-loop architecture—from traffic ingestion through detection, rule generation, and multi-node enforcement deployment—eliminates the human-in-the-loop requirement for routine threat response, enabling security teams to focus analyst attention on high-severity, novel, and complex threats requiring expert judgment.

The hybrid feature representation balances detection coverage and computational efficiency. TF-IDF character n-grams capture lexical attack signatures with evasion resistance, while behavioral statistical features provide structural characterization that resists payload-level obfuscation. The combined representation achieves strong performance without the training data requirements and inference latency of deep learning alternatives, enabling deployment on commodity hardware.

The false positive rate of 2.8% represents a substantial improvement over the 8.9% rate of the rule-based baseline, directly reducing the operational burden on security operations teams. False positive analysis identifies content management and e-commerce search use cases as primary sources of misclassification, suggesting that application-specific tuning

through the whitelist interface can further reduce FPR in production deployments.

B. Limitations

Several limitations constrain the current system:

Dataset representativeness: Evaluation on the HTTP CSIC 2010 benchmark reflects web traffic patterns from 2010, which may not fully represent modern web application request structures (JSON APIs, WebSockets, GraphQL). Production deployment would benefit from continuous training on current application traffic.

Supervised classification scope: The Random Forest classifier covers four attack categories. Modern web applications face additional attack surfaces including server-side request forgery (SSRF), insecure deserialization, and API-specific attacks that require dedicated classification models or extended feature engineering.

Adversarial robustness: While character n-gram features provide some evasion resistance, ML-based WAFs are not immune to adversarial attacks specifically designed to bypass ML detection [30]. Adaptive adversaries with knowledge of the feature extraction and model architecture could craft payloads that evade both the supervised classifier and anomaly detector.

Interpretability: Random Forest and Isolation Forest outputs lack the intuitive interpretability of signature-based WAF rules. Security analysts cannot directly inspect why a specific request was classified as malicious, potentially reducing trust in automated decisions. Integration of SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) for per-request explanations is an important direction for future work.

C. Future Research Directions

Building on the present system, several research directions merit investigation:

Transformer-based payload analysis: Pre-trained language models (BERT, CodeBERT) fine-tuned on web security corpora could capture deeper semantic context in request payloads, potentially improving detection of semantically obfuscated attacks that defeat n-gram features.

Federated learning: Privacy-preserving federated learning across multiple organizations would enable collaborative threat model training on aggregate traffic distributions unavailable to any single organization, improving coverage of rare attack variants.

Online and continual learning: Current system retraining requires periodic offline model replacement. Online learning algorithms would enable continuous model adaptation to emerging threat patterns without service interruption.

Adversarial training: Incorporating adversarially generated evasion examples into the training procedure would improve robustness against adaptive attackers.

API-specific protection: Extension to REST, GraphQL, and gRPC API semantics—including schema validation, operation depth limiting, and field-level rate limiting—would extend the system’s applicability to API-first architectures.

VII. CONCLUSION

This paper presented an AI-Powered Advanced Web Application Firewall integrating machine learning for adaptive, real-time web application protection. The system’s dual-model architecture—combining Isolation Forest anomaly detection with Random Forest threat classification on a hybrid TF-IDF and behavioral feature representation—achieves 90% accuracy, 93% precision, 92% recall, and a 2.8% false positive rate. These results represent a 68.5% FPR improvement over conventional rule-based WAF baselines while maintaining 19.5 ms end-to-end inference latency and 204,800 requests/second throughput suitable for high-traffic production environments.

The system’s containerized microservices architecture provides the horizontal scalability, fault isolation, and operational observability required for production deployment. The closed-loop design—from ML threat detection through automated rule generation and multi-node Nginx enforcement deployment—operationalizes ML-based security without requiring human intervention for routine threat response, enabling security teams to focus on complex, high-severity events requiring expert analysis.

The open-source implementation provides a reproducible reference architecture for AI-driven adaptive WAF deployment, addressing the critical gap between ML detection research and production-grade web application security systems. Future work will extend the system’s detection scope through Transformer-based semantic analysis, federated learning across organizations, and adversarial training to improve robustness against adaptive adversaries.

ACKNOWLEDGMENT

The authors express sincere gratitude to Ms. V. Swedha, M.E., (PhD), Department of Information Technology, Saveetha Engineering College, for sustained guidance throughout this research. The authors also thank Dr. G. Nagappan, Professor and Head of the Department of Computer Science and Engineering, and the Management of Saveetha Engineering College for providing the computational infrastructure supporting this work. The open-source communities behind scikit-learn, FastAPI, Nginx, Docker, Prometheus, and Grafana provided foundational tools without which this system could not have been realized.

REFERENCES

- [1] OWASP Foundation, “OWASP Top 10: 2021,” The Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] Verizon, “2023 Data Breach Investigations Report,” Verizon Business, Basking Ridge, NJ, 2023.
- [3] B. Molyneaux, *The Art of Application Performance Testing*, 2nd ed., O’Reilly Media, Sebastopol, CA, 2014.
- [4] OWASP Foundation, “OWASP ModSecurity Core Rule Set,” 2023. [Online]. Available: <https://owasp.org/www-project-modsecurity-core-rule-set/>
- [5] S. Choudhary, N. Kesswani, and D. Kumar, “Limitations of signature-based web application firewall approaches,” *Procedia Computer Science*, vol. 167, pp. 2226–2234, 2020.
- [6] A. Demetrio, G. Lagorio, G. Vella, and R. Ranise, “Waf-a-mole: Evading web application firewalls through adversarial machine learning,” in *Proc. ACM SAC*, 2021, pp. 1745–1752.
- [7] G. Apruzzese, M. Colajanni, L. Ferretti, and M. Marchetti, “The role of machine learning in cybersecurity,” *Digital Threats: Research and Practice*, vol. 3, no. 3, pp. 1–38, 2022.
- [8] R. Gonzalez-Granadillo, S. Menesidou, D. Papamartzivanos, R. Drossou, and C. Vretos, “Network security management automation: Advances, challenges, and the road ahead,” *IEEE Access*, vol. 9, pp. 83493–83553, 2021.
- [9] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, Second Quarter 2016.
- [10] C. Kruegel and G. Vigna, “Anomaly detection of web-based attacks,” in *Proc. 10th ACM CCS*, Washington, DC, 2003, pp. 251–261.
- [11] C. Torrano-Gimenez, A. Perez-Villegas, and G. Alvarez, “An anomaly-based web application firewall,” in *Proc. Int. Conf. Security and Cryptography*, 2010, pp. 14–17.
- [12] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, 2010, pp. 305–316.
- [13] D. Kozlov, T. Hieu, and G. Kasiviswanathan, “A study of web application firewall solutions,” in *Proc. IEEE Int. Conf. Smart Computing*, 2017, pp. 1–8.
- [14] A. Demetrio, A. Valenza, G. Costa, and G. Lagorio, “WAF-A-MoLE: A mutation-based approach for web application firewall evasion,” *Computers & Security*, vol. 100, p. 102087, 2021.
- [15] K. Wang, J. J. Parekh, and S. J. Stolfo, “Anagram: A content anomaly detector resistant to mimicry attack,” in *Proc. Recent Advances in Intrusion Detection (RAID)*, 2006, pp. 226–248.
- [16] R. Perdisci, D. Ariu, P. Fogla, G. Giacinto, and W. Lee, “McPAD: A multiple classifier system for accurate payload-based anomaly detection,” *Computer Networks*, vol. 53, no. 6, pp. 864–881, 2009.
- [17] Q. U. Nguyen, T. N. Nguyen, B. M. Le, and J. Ngu, “A multi-task learning framework for detecting web attacks,” *IEEE Access*, vol. 7, pp. 101954–101972, 2019.
- [18] R. C. Staudemeyer, “Applying long short-term memory recurrent neural networks to intrusion detection,” *South African Computer Journal*, vol. 56, no. 1, pp. 136–154, 2015.
- [19] X. Xie, Y. Chen, F. Bao, and C. Lim, “Security-aware transformer network for malware detection,” in *Proc. IEEE Int. Conf. Cybersecurity*, 2022, pp. 1–8.
- [20] M. Abdelnabi, A. Krombholz, and M. Fritz, “VisualPhishNet: Zero-shot phishing website detection using visual similarity,” in *Proc. ACM CCS*, 2020, pp. 1681–1698.
- [21] A. Singh and P. Verma, “Hybrid machine learning and rule-based web application firewall,” in *Proc. Int. Conf. Advanced Computing and Communication Systems (ICACCS)*, 2021, pp. 634–639.
- [22] L. Chen and T. Wu, “Anomaly detection for zero-day web attacks using autoencoders,” *Future Generation Computer Systems*, vol. 131, pp. 156–167, 2022.
- [23] S. Rahman and M. Islam, “Scalable intrusion detection with machine learning for distributed web security,” *IEEE Trans. Network and Service Management*, vol. 19, no. 3, pp. 2547–2558, 2022.
- [24] N. Patel and D. Shah, “Kubernetes-native machine learning web traffic analysis for real-time attack visibility,” *Computers & Security*, vol. 128, p. 103168, 2023.
- [25] J. Lopez and R. Martin, “Security analytics using Prometheus and Grafana for AI-powered monitoring,” in *Proc. European Conf. Systems Engineering*, 2023, pp. 1–9.

- [26] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012.
- [27] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [28] I. Csic, "HTTP CSIC 2010 Dataset for the Evaluation of Web Intrusion Detection Systems," Spanish National Research Council, Madrid, Spain, 2010.
- [29] G. Andresini, F. Palomba, and A. Appice, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. ACM AISec*, 2021, pp. 111–122.
- [30] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proc. ACM ASIA CCS*, 2017, pp. 506–519.
- [31] Sriram V, Surothaaman R, and Pavithra M, "AI-Powered Advanced Web Application Firewall — Source Code Repository," GitHub, 2025. [Online]. Available: <https://github.com/Darkwebnew/Projectwork1.git>

Sriram V is a final-year undergraduate student in Computer Science and Engineering (Cybersecurity) at Saveetha Engineering College, Chennai, India. His research interests include machine learning for cybersecurity, web application security, and intelligent intrusion detection systems.

Surothaaman R is a final-year undergraduate student in Computer Science and Engineering (Cybersecurity) at Saveetha Engineering College, Chennai, India. His research interests include network security, anomaly detection, and AI-driven security systems.

Pavithra M is a final-year undergraduate student in Computer Science and Engineering (Cybersecurity) at Saveetha Engineering College, Chennai, India. Her research interests include web application security, machine learning, and containerized deployment architectures.

Swedha V is an Assistant Professor in the Department of Information Technology, Saveetha Engineering College, Chennai, India. She holds a M.E. degree and is currently pursuing her PhD. Her research interests span cybersecurity, artificial intelligence applications in security, and network intrusion detection.